TALENTFORGE ECE POC — PROOF OF CONCEPT

Electronics & Electrical Engineering Domain

10-Week Sprint to Validated Product-Market Fit

EXECUTIVE SUMMARY FOR EXECUTION

POC Objective

Build a **fully operational ECE talent marketplace** that proves: ✅ ECE students can complete circuit design projects in a sandbox environment
✅ AI validates circuit simulations automatically (Proteus, MATLAB, LTSpice)
✅ Employers can discover and evaluate verified ECE talent
✅ Psychology matching improves hiring success by 35%+
✅ Unit economics work (6:1 LTV:CAC achieved)

Why ECE First?

- **Untapped Market**: 800K+ ECE graduates/year in India, ZERO platforms serve them
- **Technical Differentiation**: Automatic circuit validation via SPICE simulation is our IP
- **Employer Demand**: Electronics/IoT startups desperately need verified junior engineers
- **Lower Complexity**: ECE challenges are scoped better than mechanical (simulations are faster)
- **Investor Appeal**: "We validated electronics talent in 10 weeks, now scaling to all branches"

Success Criteria (Week 10)

- 300 ECE students registered
- 50 ECE employers onboarded
- 200+ completed circuit projects
- ₹25 lakhs MRR (Month 10 run-rate)
- 4.3+ NPS score (students & employers)
- 5 case studies with testimonials
- 8 college partnerships signed

POC Investment: ₹38 lakhs (lean, focused) **Path to Series A**: Demonstrate 15x growth trajectory by Month 3

PART 1: TECHNICAL ARCHITECTURE FOR ECE

A. Core Tech Stack (ECE-Specific)

Backend (API Layer)

```
Framework: Node.js + Express (for speed & flexibility)
Database: PostgreSQL (primary) + Redis (caching)
Real-time: WebSocket for live simulation status
File Storage: AWS S3 (circuit files, simulation outputs)
Job Queue: Bull/RabbitMQ (async validation jobs)
Authentication: JWT + OAuth (Google, Microsoft)
```

Frontend

```
Web: React.js + Tailwind CSS
Mobile: React Native (iOS/Android, later phase)
Interactive Circuits: Draw.io API (circuit diagram preview)
3D Visualization: Three.js (PCB layout visualization)
```

ECE Simulation Engine (CRITICAL)

```
Primary: Proteus (via cloud API or headless SPICE)
Secondary: PySpice (open-source Python wrapper)
Tertiary: LTSpice (free, powerful, headless-friendly)
Alternative: ngspice (fully open-source fallback)

MATLAB/Octave integration for signal processing validation
Jupyter notebooks for student walkthrough & documentation
```

AI/ML Pipeline

```
Circuit validation: Custom Python validators for SPICE output
Psychology assessment: Big Five + Technical Aptitude + Work Style
Matching engine: XGBoost models (skill-to-project matching)
Anomaly detection: Identify suspicious submissions (copy-paste)
```

Blockchain (Minimal MVP)

```
Network: Polygon (low gas, ₹0.01 per transaction)
```

```
Smart Contracts: ERC-721 NFT badges for project completion
Wallet: Torus (Aadhaar-linked, one-click onboarding for India)
Storage: IPFS (circuit files, simulation reports)
```

B. ECE-Specific Infrastructure Components

Component 1: Circuit File Ingestion & Validation Engine

Supported Formats:

- `.ckt` (Proteus format) — primary
- `.cir` (SPICE netlist) — universal
- `.asc` (LTSpice schematic) — fallback
- `.png` (Circuit diagram image, for preview only)

Python Validation Pipeline:

```python
```

```python
# talentforge_ece/circuit_validator.py

import os
import subprocess
import json
import re
from typing import Dict, List, Tuple

class CircuitValidator:
    """
    Validates ECE student circuit submissions against project specs
    Runs SPICE simulations, extracts key metrics, scores quality
    """

    def __init__(self, circuit_file_path: str, project_spec: Dict):
        self.circuit_file = circuit_file_path
        self.spec = project_spec  # Acceptance criteria
        self.results = {}
        self.simulation_output = None

    # ============ STEP 1: Parse Circuit ============
    def parse_circuit_netlist(self) -> bool:
        """
        Convert Proteus .ckt to SPICE netlist
        Or validate existing .cir netlist
        """
        try:
            if self.circuit_file.endswith('.ckt'):
                # Proteus to SPICE conversion (via Proteus CLI or manual parsing)
                netlist = self._proteus_to_spice(self.circuit_file)
            elif self.circuit_file.endswith('.cir'):
                netlist = self._read_spice_netlist(self.circuit_file)
            else:
                raise ValueError(f"Unsupported format: {self.circuit_file}")

            self.netlist = netlist
            self.results['netlist_valid'] = True
            return True

        except Exception as e:
            self.results['netlist_valid'] = False
            self.results['parse_error'] = str(e)
            return False
```

```
    def _read_spice_netlist(self, filepath: str) -> str:
        """Read and validate SPICE netlist syntax"""
        with open(filepath, 'r') as f:
            lines = f.readlines()

        # Basic SPICE validation
        required_keywords = ['.title', '.subckt', '.end', '.tran', '.ac']
        has_required = any(any(kw in line.lower() for kw in required_keywords) for li

        if not has_required:
            raise ValueError("Invalid SPICE netlist: missing required directives")

        return ''.join(lines)

    def _proteus_to_spice(self, proteus_file: str) -> str:
        """
        Proteus .ckt to SPICE conversion
        Option 1: Call Proteus headless CLI
        Option 2: Parse XML structure of .ckt file
        """
        # Simplified: assume Proteus CLI available
        temp_netlist = '/tmp/converted.cir'
        cmd = f'proexe -export {proteus_file} {temp_netlist}'

        try:
            subprocess.run(cmd, shell=True, check=True, timeout=10)
            return self._read_spice_netlist(temp_netlist)
        except:
            raise ValueError("Proteus conversion failed. Ensure Proteus is installed.

    # ============ STEP 2: Run Simulation ============
    def run_spice_simulation(self) -> Dict:
        """
        Execute SPICE simulation (using ngspice or PySpice)
        Extract voltage, current, frequency response data
        """
        import PySpice.Logging.Logging as Logging
        from PySpice.Spice.Netlist import Circuit
        from PySpice.Unit import *
        from PySpice.Spice.NgSpice.Shared import NgSpiceShared

        try:
            # Create simulator
```

```
        simulator = Circuit(self.netlist)

        # Choose simulation type based on project spec
        if self.spec.get('sim_type') == 'transient':
            # Time-domain analysis
            analysis = simulator.simulator(temperature=25, nominal_temperature=25
            plot = analysis.transient(step_time=self.spec['step_time'],
                                      end_time=self.spec['end_time'])

        elif self.spec.get('sim_type') == 'ac':
            # Frequency response (Bode plot)
            analysis = simulator.simulator()
            plot = analysis.ac(start_frequency=self.spec['start_freq']@u_Hz,
                               stop_frequency=self.spec['stop_freq']@u_Hz,
                               number_of_points=self.spec['points'],
                               variation='dec')  # Logarithmic sweep

        elif self.spec.get('sim_type') == 'dc':
            # DC sweep analysis
            analysis = simulator.simulator()
            plot = analysis.dc(v_sweep_source=self.spec['sweep_var'],
                               v_start=self.spec['v_start'],
                               v_stop=self.spec['v_stop'],
                               v_step=self.spec['v_step'])

        # Extract results
        self.simulation_output = {
            'success': True,
            'time': plot.time.as_ndarray() if hasattr(plot, 'time') else None,
            'voltages': {node: plot[node].as_ndarray() for node in plot.nodes},
            'currents': {branch: plot[branch].as_ndarray() for branch in plot.bra
        }

        return self.simulation_output

    except Exception as e:
        self.simulation_output = {
            'success': False,
            'error': str(e)
        }
        return self.simulation_output

# ============ STEP 3: Validate Against Specs ============
def validate_against_specs(self) -> Dict:
```

```
        """
        Check if simulation results meet project acceptance criteria
        Example specs:
        {
          'output_voltage_peak': {'min': 4.5, 'max': 5.5},  # volts
          'ripple': {'max': 0.1},  # %
          'frequency_response': {'peak': -3, 'at_freq': 1000},  # dB, Hz
          'efficiency': {'min': 85}  # %
        }
        """
        if not self.simulation_output or not self.simulation_output['success']:
            return {'passed': False, 'reason': 'Simulation failed'}

        validation = {'passed': True, 'criteria_met': {}, 'margin_of_safety': {}}

        voltages = self.simulation_output['voltages']

        # Check output voltage range
        if 'output_voltage_peak' in self.spec:
            peak_volt = max(voltages.get('Vout', [0]))
            spec_range = self.spec['output_voltage_peak']
            meets = spec_range['min'] <= peak_volt <= spec_range['max']
            margin = ((spec_range['max'] - peak_volt) / spec_range['max']) * 100

            validation['criteria_met']['output_voltage'] = meets
            validation['margin_of_safety']['output_voltage'] = margin

            if not meets:
                validation['passed'] = False

        # Check ripple voltage
        if 'ripple' in self.spec and 'Vout' in voltages:
            vout = voltages['Vout']
            ripple_pct = ((max(vout) - min(vout)) / max(vout)) * 100 if max(vout) > 0

            meets = ripple_pct <= self.spec['ripple']['max']
            validation['criteria_met']['ripple'] = meets
            validation['margin_of_safety']['ripple'] = self.spec['ripple']['max'] - r

            if not meets:
                validation['passed'] = False

        return validation
```

```python
    # ============ STEP 4: Grade Quality ============
    def calculate_quality_score(self) -> int:
        """
        Score 0-100 based on:
        - Simulation success
        - Specification compliance
        - Efficiency/margin of safety
        - Code quality (netlist cleanliness)
        """
        score = 100

        # Simulation success
        if not self.simulation_output or not self.simulation_output['success']:
            return 0

        # Spec compliance
        validation = self.validate_against_specs()
        if not validation['passed']:
            score -= 30

        # Margin of safety (how much headroom from spec limits)
        avg_margin = sum(validation.get('margin_of_safety', {}).values()) / max(len(v
        if avg_margin < 5:
            score -= 15  # Too close to limits
        elif avg_margin < 10:
            score -= 8

        # Netlist quality (cleanliness, comments, organization)
        netlist_quality = self._score_netlist_quality()
        score -= (10 - netlist_quality)  # Max 10 points deduction

        return max(0, score)

    def _score_netlist_quality(self) -> int:
        """
        Rate netlist on:
        - Proper formatting
        - Component naming conventions
        - Comments & documentation
        - No unnecessary nodes
        """
        score = 10
        lines = self.netlist.split('\n')
```

```
        # Check for comments
        has_comments = any(';' in line or '*' in line for line in lines)
        if not has_comments:
            score -= 2

        # Check component naming (R1, R2, C1, L1, etc. — proper convention)
        component_lines = [l for l in lines if l and l[0] in 'RCLEDVIMS']
        proper_naming = sum(1 for l in component_lines if self._check_naming_conventi
        if len(component_lines) > 0:
            naming_ratio = proper_naming / len(component_lines)
            if naming_ratio < 0.8:
                score -= 2

        # Check for excessive nodes (indicates inefficient design)
        node_count = len(set(re.findall(r'N\d+|GND|Vdd|Vcc', self.netlist)))
        if node_count > 20:  # Arbitrary threshold
            score -= 1

        return max(0, score)

    def _check_naming_convention(self, line: str) -> bool:
        """Check if component follows standard naming (R1, C2, etc.)"""
        parts = line.split()
        if not parts:
            return False
        name = parts[0]
        # R/C/L/M/D should be followed by number
        if name[0] in 'RCLDM' and len(name) > 1 and name[1:].isdigit():
            return True
        return False

    # ============ STEP 5: Generate Report ============
    def generate_validation_report(self) -> Dict:
        """
        Return comprehensive validation report for storage & display
        """
        return {
            'netlist_valid': self.results.get('netlist_valid', False),
            'simulation_success': self.simulation_output['success'] if self.simulatio
            'validation': self.validate_against_specs(),
            'quality_score': self.calculate_quality_score(),
            'simulation_data': {
                'voltages_summary': {k: {'min': float(min(v)), 'max': float(max(v))}
                                     for k, v in (self.simulation_output.get('voltages'
```

```
                'currents_summary': {k: {'min': float(min(v)), 'max': float(max(v))}
                                     for k, v in (self.simulation_output.get('currents
            },
            'timestamp': __import__('datetime').datetime.now().isoformat()
        }

    # ============ COMPLETE WORKFLOW ============
    def validate_complete(self) -> Tuple[bool, int, Dict]:
        """
        End-to-end validation: parse → simulate → validate → score
        Returns: (passed, quality_score, detailed_report)
        """
        if not self.parse_circuit_netlist():
            return False, 0, {'error': self.results.get('parse_error')}

        if not self.run_spice_simulation():
            return False, 0, {'error': 'Simulation failed'}

        report = self.generate_validation_report()
        passed = report['validation'].get('passed', False)
        score = report['quality_score']

        return passed, score, report


# ============ USAGE EXAMPLE ============
"""
validator = CircuitValidator(
    '/uploads/student_circuits/filter_design.cir',
    project_spec={
        'sim_type': 'ac',
        'start_freq': 10,
        'stop_freq': 100000,
        'points': 100,
        'output_voltage_peak': {'min': 4.5, 'max': 5.5},
        'ripple': {'max': 0.1},
        'frequency_response': {'peak': -3, 'at_freq': 1000}
    }
)

passed, score, report = validator.validate_complete()
print(f"Passed: {passed}, Score: {score}/100")
```

```
print(json.dumps(report, indent=2))
"""
```

Key Features:

- ✅ Automatic SPICE simulation (no manual QA)
- ✅ Real-time circuit validation
- ✅ Quality scoring (0-100)
- ✅ Spec compliance checking
- ✅ Margin of safety calculation
- ✅ Component naming validation
- ✅ Timestamped reports

Component 2: ECE Project Templates (30 Ready-to-Deploy)

Tier 1 (Beginner): 10 Projects

- Resistor color code calculator
- Basic Ohm's law circuit analysis
- LED circuit design (5V, 20mA constraint)
- Voltage divider circuit
- Series/parallel resistor networks
- Capacitor charging/discharging curves
- Simple RC low-pass filter
- Transistor switch (digital logic)
- 555 Timer astable oscillator
- Power supply ripple analysis

Tier 2 (Intermediate): 12 Projects

- Active low-pass filter (Sallen-Key)
- Op-amp summing amplifier
- Precision rectifier circuit
- Class A amplifier design
- Phase-shift oscillator
- Comparator hysteresis circuit
- PWM controller for motor

- EMI filter design (power electronics)
- Signal conditioning for ADC
- Wien bridge oscillator
- Integrator/differentiator circuits
- Audio amplifier (BJT-based)

Tier 3 (Advanced): 8 Projects

- Switching power supply design
- PID controller circuit
- RF filter design (100 MHz+)
- Mixed-signal circuit (analog + digital)
- Sensor signal conditioning
- Phase-locked loop (PLL) design
- Buck converter efficiency analysis
- Multi-stage amplifier with feedback

Each Project Includes:

```json
```

```
{
  "project_id": "ece_tier2_006",
  "title": "Active Low-Pass Filter Design",
  "tier": 2,
  "duration_days": 21,
  "stipend_rupees": 8000,
  "difficulty": "Intermediate",

  "tools_required": ["Proteus", "MATLAB"],
  "skills": ["circuit_design", "filter_theory", "op_amps", "simulation"],

  "project_brief": "Design a 2nd-order Sallen-Key active Butterworth low-pass filter.

  "acceptance_criteria": {
    "cutoff_frequency": {"value": 1000, "unit": "Hz", "tolerance": 0.05},
    "passband_ripple": {"max": 0.1, "unit": "dB"},
    "attenuation_10khz": {"min": 40, "unit": "dB"},
    "quality_factor": {"value": 0.707, "tolerance": 0.1},
    "simulation_accuracy": {"min": 0.95}
  },

  "deliverables": [
    "Circuit schematic (Proteus .ckt or .png)",
    "SPICE netlist (.cir)",
    "Bode plot (magnitude & phase)",
    "Component values & justification",
    "5-page technical report"
  ],

  "rubric": {
    "circuit_correctness": 25,
    "spec_compliance": 25,
    "analysis_quality": 25,
    "documentation": 15,
    "code_cleanliness": 10
  },

  "peer_leaderboard": true,
  "estimated_xp": 200,
  "prerequisites": ["ece_tier1_007", "ece_tier1_008"]
}
```

Component 3: Psychology + ECE Matching Engine

ECE-Specific Personality Dimensions:

```python
```

```python
def calculate_ece_project_fit(student_profile: Dict, project: Dict) -> float:
    """
    Matches student psychology to ECE project requirements
    Returns compatibility score 0-100
    """

    score = 0
    weights = {
        'analytical_thinking': 0.25,
        'attention_to_detail': 0.25,
        'problem_solving_style': 0.20,
        'hands_on_preference': 0.15,
        'creativity': 0.15
    }

    # 1. Analytical Thinking
    # ECE requires strong analytical skills for circuit analysis
    student_analytical = student_profile.get('analytical_thinking', 50)
    project_analytical_demand = project.get('analytical_demand', 50)  # 0-100

    analytical_fit = 100 - abs(student_analytical - project_analytical_demand)
    score += analytical_fit * weights['analytical_thinking']

    # 2. Attention to Detail
    # Filter design, circuit optimization demand precision
    student_detail = student_profile.get('conscientiousness', 50)
    project_detail_demand = project.get('precision_required', 50)

    detail_fit = 100 - abs(student_detail - project_detail_demand)
    score += detail_fit * weights['attention_to_detail']

    # 3. Problem-Solving Style
    # Theoretical vs hands-on trade-off
    student_hands_on = student_profile.get('hands_on_vs_theoretical', 50)
    project_hands_on = project.get('hands_on_requirement', 50)

    hands_on_fit = 100 - abs(student_hands_on - project_hands_on)
    score += hands_on_fit * weights['problem_solving_style']

    # 4. Hands-On Preference
    # Does student prefer simulation or actual breadboard work?
    student_simulator = student_profile.get('prefers_simulation', 50)
    project_simulator_ratio = project.get('simulation_ratio', 50)
```

```
    simulator_fit = 100 - abs(student_simulator - project_simulator_ratio)
    score += simulator_fit * weights['hands_on_preference']

    # 5. Creativity
    # Some ECE projects (oscillator design) need creative thinking
    student_creativity = student_profile.get('openness', 50)
    project_creativity_demand = project.get('creativity_required', 50)

    creativity_fit = 100 - abs(student_creativity - project_creativity_demand)
    score += creativity_fit * weights['creativity']

    return round(score, 1)

# Example output:
# Student: Analytical (85), Detail-oriented (78), Theory-focused (65)
# Project: Active Filter (analytical 80, precision 75, theory 70)
# Match Score: 82/100 ✅ RECOMMENDED
```

Component 4: Blockchain NFT Issuance for ECE Badges

Smart Contract (Solidity):

```solidity
```

```solidity
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract TalentForgeECEBadges is ERC721, Ownable {

    struct ECEBadge {
        uint256 projectId;
        address student;
        uint256 qualityScore;      // 0-100
        string projectTitle;
        string projectDomain;      // "analog_design", "digital_circuits", etc.
        uint256 issuedAt;
        string ipfsCID;            // IPFS hash for circuit files & report
        bool passed;
    }

    mapping(uint256 => ECEBadge) public badges;
    mapping(address => uint256[]) public studentBadges;

    uint256 private tokenCounter = 0;

    event BadgeMinted(
        uint256 indexed tokenId,
        address indexed student,
        string projectTitle,
        uint256 qualityScore
    );

    function issueBadge(
        uint256 projectId,
        address student,
        uint256 qualityScore,
        string memory projectTitle,
        string memory projectDomain,
        string memory ipfsCID,
        bool passed
    ) external onlyOwner returns (uint256) {

        tokenCounter++;
        uint256 tokenId = tokenCounter;
```

```
        badges[tokenId] = ECEBadge({
            projectId: projectId,
            student: student,
            qualityScore: qualityScore,
            projectTitle: projectTitle,
            projectDomain: projectDomain,
            issuedAt: block.timestamp,
            ipfsCID: ipfsCID,
            passed: passed
        });

        studentBadges[student].push(tokenId);

        _safeMint(student, tokenId);

        emit BadgeMinted(tokenId, student, projectTitle, qualityScore);

        return tokenId;
    }

    function getBadgeMetadata(uint256 tokenId)
        external
        view
        returns (ECEBadge memory) {
        return badges[tokenId];
    }

    function getStudentBadges(address student)
        external
        view
        returns (uint256[] memory) {
        return studentBadges[student];
    }

    function tokenURI(uint256 tokenId)
        public
        view
        override
        returns (string memory) {

        ECEBadge memory badge = badges[tokenId];

        string memory json = string(abi.encodePacked(
            '{"name":"', badge.projectTitle, '",',
```

```
            '"description":"ECE project completed on TalentForge",',
            '"image":"ipfs://', badge.ipfsCID, '/badge.png",',
            '"attributes":[',
            '{"trait_type":"Quality Score","value":', uint2str(badge.qualityScore), '
            '{"trait_type":"Domain","value":"', badge.projectDomain, '"},',
            '{"trait_type":"Passed","value":', badge.passed ? 'true' : 'false', '}',
            ']}'
        ));

        return string(abi.encodePacked(
            'data:application/json;base64,',
            base64Encode(bytes(json))
        ));
    }

    function uint2str(uint256 _i) internal pure returns (string memory) {
        if (_i == 0) return "0";
        uint256 j = _i;
        uint256 len;
        while (j != 0) { len++; j /= 10; }
        bytes memory bstr = new bytes(len);
        uint256 k = len;
        while (_i != 0) {
            k = k-1;
            uint8 temp = (48 + uint8(_i - _i / 10 * 10));
            bytes1 b1 = bytes1(temp);
            bstr[k] = b1;
            _i /= 10;
        }
        return string(bstr);
    }

    function base64Encode(bytes memory data) internal pure returns (string memory) {
        // Simple base64 encoding (production should use external library)
        bytes memory TABLE = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz012
        // ... encoding logic ...
        return "";
    }
}
```

NFT Badge Example (Minted on Polygon):

```
json
```

```
{
  "tokenId": 4521,
  "name": "Active Low-Pass Filter Design",
  "description": "ECE intermediate project completed on TalentForge",
  "image": "ipfs://QmXxxx.../badge.png",
  "attributes": [
    {
      "trait_type": "Quality Score",
      "value": 87
    },
    {
      "trait_type": "Domain",
      "value": "analog_filter_design"
    },
    {
      "trait_type": "Passed",
      "value": true
    },
    {
      "trait_type": "Issued Date",
      "value": "2024-02-15"
    }
  ],
  "contractAddress": "0x7f8A...ECEBadges",
  "blockchain": "Polygon",
  "transactionHash": "0x9d2c...a8f"
}
```

PART 2: 10-WEEK PROJECT TIMELINE

PHASE STRUCTURE

```
Week 1:     Setup & Architecture
Week 2-3:   Backend Core + DB
Week 4-5:   Circuit Validator & Simulation
Week 6-7:   Frontend + Gamification
Week 8:     Beta Testing (50 users)
Week 9:     Polish & Optimization
Week 10:    Public Launch + Investor Demo
```

DETAILED WEEK-BY-WEEK BREAKDOWN

WEEK 1: Foundation & Setup

Monday-Tuesday: Infrastructure

Deliverables:

- [ ] AWS account setup (EC2, RDS, S3, Lambda)
- [ ] Polygon Mumbai testnet account + smart contract wallet
- [ ] GitHub repos created (backend, frontend, smart-contracts)
- [ ] Slack + monitoring tools configured (DataDog, Sentry)
- [ ] Database schema designed and documented
- [ ] Tech stack finalized (Node.js, PostgreSQL, React)

Team Assignment:

- 1 DevOps engineer: AWS + Infrastructure
- 1 Backend lead: Database schema review
- 1 Full-stack engineer: Git repo setup

Success Metrics:

- AWS console accessible, EC2 instance running
- GitHub has 3 repos with proper .gitignore
- Team can push code and pass CI checks

Wednesday-Friday: Architecture & Design

Deliverables:

- [ ] System architecture diagram (Figma/Miro)
  - Student flow: Register → Assessment → Project → Submission → Validation → NFT
  - Company flow: Post project → Review submissions → Hire
- [ ] Database schema finalized (DDL scripts)
  - students, projects, submissions, employers, nft_badges, psychometric_profiles
- [ ] API specification (OpenAPI/Swagger)
  - 40+ endpoints mapped out
  - Request/response schemas defined
- [ ] Security & authentication strategy document

- JWT token flow
- OAuth integration (Google, Microsoft)
- Rate limiting, CORS policy

Team Assignment:

- 1 Architect/Tech Lead: Overall design
- 1 Backend engineer: API spec
- 1 Full-stack engineer: Database & schema

Success Metrics:

- Architecture diagram is clear and signed off
- API spec covers all 4 user flows (student, employer, admin, evaluator)
- Database schema normalized and indexed
- Security review checklist passed

WEEK 2-3: Backend Core + Database

Week 2: Authentication & User Management

Deliverables:

Day 1-2:

- [ ] User authentication endpoints
  - POST /auth/register (email + password, or OAuth)
  - POST /auth/login
  - POST /auth/refresh-token
  - POST /auth/logout
- [ ] User model & database migration
- [ ] JWT token generation & validation middleware
- [ ] Password hashing (bcrypt) + salt management

Day 3-5:

- [ ] Psychometric assessment API
  - POST /students/psychometric/start (begin 30-min test)
  - GET /students/psychometric/question/:number
  - POST /students/psychometric/submit
  - GET /students/psychometric/results (scores returned)

- [ ] Assessment scoring algorithm
  - Big Five calculation
  - Work style assessment
  - Technical aptitude scoring
- [ ] Profile storage in database

Code Example (Express.js):

```javascript
```

```
// routes/auth.js
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const router = express.Router();

router.post('/register', async (req, res) => {
  const { email, password, name, domain } = req.body;

  // Validate email format
  if (!email.match(/^[^\s@]+@[^\s@]+\.[^\s@]+$/)) {
    return res.status(400).json({ error: 'Invalid email' });
  }

  // Check if user exists
  const existing = await Student.findOne({ email });
  if (existing) {
    return res.status(409).json({ error: 'User already exists' });
  }

  // Hash password
  const salt = await bcrypt.genSalt(10);
  const hashedPassword = await bcrypt.hash(password, salt);

  // Create student record
  const student = new Student({
    email,
    name,
    domain,  // 'ece', 'eee', 'mech', etc.
    passwordHash: hashedPassword,
    tier: 'Trainee',
    totalXP: 0,
    createdAt: new Date()
  });

  await student.save();

  // Issue JWT token
  const token = jwt.sign(
    { userId: student._id, email: student.email, tier: student.tier },
    process.env.JWT_SECRET,
    { expiresIn: '24h' }
  );
```

```
  res.status(201).json({
    token,
    student: {
      id: student._id,
      email: student.email,
      name: student.name,
      domain: student.domain,
      tier: student.tier
    }
  });
});

module.exports = router;
```

Psychometric Assessment Scoring:

javascript

```
// services/psychometric.js
function calculateBigFive(answers) {
  // answers = { q1: 4, q2: 2, ... q50: 5 } (Likert scale 1-5)

  const dimensions = {
    openness: [1, 6, 11, 16, 21, 26, 31, 36, 41, 46],      // Q numbers
    conscientiousness: [2, 7, 12, 17, 22, 27, 32, 37, 42, 47],
    extraversion: [3, 8, 13, 18, 23, 28, 33, 38, 43, 48],
    agreeableness: [4, 9, 14, 19, 24, 29, 34, 39, 44, 49],
    neuroticism: [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
  };

  const profile = {};

  for (const [dim, questions] of Object.entries(dimensions)) {
    const scores = questions.map(q => answers[`q${q}`] || 3);
    const avg = (scores.reduce((a, b) => a + b) / scores.length) * 20;  // Scale 0-10
    profile[dim] = Math.round(avg);
  }

  return profile;
}

// Example output:
// { openness: 72, conscientiousness: 88, extraversion: 45, agreeableness: 78, neurot
```

Success Metrics:

- User can register and receive JWT token
- OAuth login works (Google, Microsoft)
- 30+ users complete psychometric test
- Assessment scores stored and retrievable
- Authentication on all protected routes working

Week 3: Projects & Submissions API

Deliverables:

Day 1-2:

- [ ] Project listing endpoints

- GET /projects (with filters: tier, domain, difficulty)
- GET /projects/:id (detailed project view)
- POST /employers/projects (create new project, employer only)

☐ Project data model in database

☐ Tier unlock logic

- Tier 1 (Trainee): Free projects
- Tier 2 (Contributor): Unlock after completing 2 Tier 1 projects
- Tier 3 (Associate): Unlock after completing 2 Tier 2 projects

Day 3-5:

☐ Submission endpoints

- POST /projects/:id/submit (upload circuit file + metadata)
- GET /submissions/:id (view submission status)
- PUT /submissions/:id/withdraw (retract submission)

☐ File upload to S3

- Support .cir, .ckt, .asc formats
- Virus scanning before storage
- File versioning

Code Example:

```javascript
```

```
// routes/projects.js
router.get('/projects', authenticateToken, async (req, res) => {
  const { tier = 'all', domain = 'ece', difficulty = 'all' } = req.query;
  const student = await Student.findById(req.user.id);

  // Filter by tier unlock
  let tierFilter = { requiredTier: 'Trainee' };
  if (student.tier === 'Contributor') {
    tierFilter = { requiredTier: { $in: ['Trainee', 'Contributor'] } };
  }
  if (student.tier === 'Associate') {
    tierFilter = { requiredTier: { $in: ['Trainee', 'Contributor', 'Associate'] } };
  }

  let query = { domain, ...tierFilter };

  if (difficulty !== 'all') {
    query.difficulty = difficulty;
  }

  const projects = await Project.find(query)
    .select('id title difficulty stipendRupees duration assessment')
    .lean();

  // Add psychology match score for each project
  const enrichedProjects = projects.map(proj => ({
    ...proj,
    psychologyMatchScore: calculateProjectFit(student.psychometricProfile, proj),
    matchReason: generateMatchExplanation(student.psychometricProfile, proj)
  }));

  res.json(enrichedProjects);
});

// POST submission
router.post('/projects/:projectId/submit', authenticateToken, upload.single('circuitF
  const { projectId } = req.params;
  const student = await Student.findById(req.user.id);
  const project = await Project.findById(projectId);

  // Check tier eligibility
  if (!canUnlockTier(student.tier, project.requiredTier)) {
    return res.status(403).json({ error: 'Tier not unlocked yet' });
```

```
  }

  // Upload circuit file to S3
  const s3Key = `circuits/${student._id}/${projectId}_${Date.now()}.cir`;
  const uploadParams = {
    Bucket: process.env.AWS_S3_BUCKET,
    Key: s3Key,
    Body: req.file.buffer,
    ContentType: 'application/octet-stream'
  };

  const s3Result = await s3.upload(uploadParams).promise();

  // Create submission record
  const submission = new Submission({
    studentId: student._id,
    projectId: project._id,
    circuitFilePath: s3Key,
    s3Url: s3Result.Location,
    submittedAt: new Date(),
    status: 'submitted'
  });

  await submission.save();

  // Queue async validation job
  await validationQueue.add('validate_circuit', {
    submissionId: submission._id,
    circuitUrl: s3Result.Location,
    projectSpec: project.acceptanceCriteria
  }, { delay: 1000 });  // Start validation after 1 second

  res.status(201).json({
    submissionId: submission._id,
    status: 'submitted',
    estimatedValidationTime: '5 minutes'
  });
});
```

Success Metrics:

- 30 ECE projects loaded into database
- Students can browse projects by tier

- File upload to S3 working
- Submission records created successfully
- Async queue processes validations

WEEK 4-5: Circuit Validator & Simulation Engine

Week 4: SPICE Integration & Circuit Parsing

Deliverables:

Day 1-2: Proteus/SPICE Integration

- [ ] Docker container with ngspice installed
- [ ] PySpice Python wrapper configured
- [ ] Circuit file format conversion (Proteus → SPICE netlist)
- [ ] Test validation on 10 sample circuits

Docker Setup:

```dockerfile
```

```
# Dockerfile for ECE validation service
FROM python:3.9-slim

RUN apt-get update && apt-get install -y \
    ngspice \
    graphviz \
    git

WORKDIR /app

COPY requirements.txt .
RUN pip install -q \
    PySpice==1.5 \
    numpy==1.24.3 \
    scipy==1.10.1 \
    matplotlib==3.7.1 \
    redis==4.5.5 \
    boto3==1.26.142 \
    python-dotenv==1.0.0

COPY . .

CMD ["python", "-u", "validation_worker.py"]
```

Day 3-5: Async Validation Pipeline

- [ ] Redis job queue (Bull/RabbitMQ wrapper)
- [ ] Validation worker processes submissions
- [ ] Simulation timeout handling (30 sec max)
- [ ] Error logging & retry logic

Code Example (Async Worker):

```
python
```

```python
# talentforge_ece/validation_worker.py

import json
import logging
import time
import boto3
from redis import Redis
from circuit_validator import CircuitValidator
from database import Submission, get_mongodb_connection

logger = logging.getLogger(__name__)
redis_client = Redis(host='localhost', port=6379, db=0)
s3_client = boto3.client('s3')
db = get_mongodb_connection()

def process_validation_job():
    """
    Worker process that continuously pulls validation jobs from queue
    Runs SPICE simulation, validates, scores, and stores results
    """

    while True:
        try:
            # Pop job from queue (blocking, timeout 5 sec)
            job = redis_client.blpop('validation_queue', timeout=5)

            if not job:
                continue

            _, job_data = job
            job_json = json.loads(job_data)
            submission_id = job_json['submission_id']
            circuit_s3_url = job_json['circuit_s3_url']
            project_spec = job_json['project_spec']

            logger.info(f"Processing validation for submission {submission_id}")

            # Update submission status
            submission = db.submissions.find_one_and_update(
                {'_id': submission_id},
                {'$set': {'status': 'validating', 'validationStartedAt': time.time()}
                return_document=True
            )
```

```
            # Download circuit file from S3
            temp_filepath = f'/tmp/{submission_id}.cir'
            s3_client.download_file(
                Bucket=os.environ['AWS_S3_BUCKET'],
                Key=circuit_s3_url.split('/')[-1],
                Filename=temp_filepath
            )

            # Validate circuit
            validator = CircuitValidator(temp_filepath, project_spec)
            passed, quality_score, report = validator.validate_complete()

            # Store validation results
            db.submissions.update_one(
                {'_id': submission_id},
                {
                    '$set': {
                        'status': 'validated',
                        'qualityScore': quality_score,
                        'validationResults': report,
                        'validationCompleted': True,
                        'validationTimestamp': time.time()
                    }
                }
            )

            logger.info(f"✓ Submission {submission_id} validated: Score={quality_scor

            # If score > 75, mint NFT badge
            if quality_score >= 75:
                mint_nft_badge(submission_id, quality_score, report)

        except Exception as e:
            logger.error(f"Validation error: {str(e)}")
            db.submissions.update_one(
                {'_id': submission_id},
                {'$set': {'status': 'failed', 'error': str(e)}}
            )
            time.sleep(1)  # Wait before retrying

if __name__ == '__main__':
```

```
    logger.info("ECE Circuit Validator Worker Started")
    process_validation_job()
```

Day 4-5: Validation Pipeline Testing

- [ ] Test with 20 valid circuits
- [ ] Test with 10 invalid circuits (syntax errors)
- [ ] Test timeout handling (>30 sec simulations)
- [ ] Measure average validation time (target: <5 min)

Success Metrics:

- Validation queue processes 5+ submissions/minute
- Valid circuits get accurate scores
- Invalid circuits caught and reported
- Average validation time < 5 minutes
- Worker handles errors gracefully (no crashes)

Week 5: Full Validation & Scoring Pipeline

Deliverables:

Day 1-2: Complete Validation Workflow

- [ ] Circuit parsing ✅ (Week 4)
- [ ] SPICE simulation ✅ (Week 4)
- [ ] Spec compliance checking
  - Compare simulation output to acceptance criteria
  - Calculate margin of safety
  - Grade quality (0-100)
- [ ] Generate detailed validation report

Day 3-4: Quality Scoring

- [ ] Scoring algorithm:
  - Netlist quality (10%)
  - Specification compliance (60%)
  - Margin of safety (20%)
  - Innovation/optimization (10%)
- [ ] Score breakdown returned to student

☐ Feedback explanations

Scoring Example:

```python
```

```python
def score_submission(validation_report):
    """
    Generate final quality score (0-100)
    Breaks down into categories for student feedback
    """

    score = {
        'total': 0,
        'breakdown': {},
        'feedback': []
    }

    # 1. Netlist Quality (10 pts)
    netlist_quality = validation_report['netlist_quality_score']  # 0-10
    score['breakdown']['netlist'] = netlist_quality
    score['total'] += netlist_quality

    if netlist_quality < 5:
        score['feedback'].append('Consider reorganizing your netlist for clarity. Add

    # 2. Specification Compliance (60 pts)
    criteria_met = validation_report['validation']['criteria_met']
    criteria_count = len(criteria_met)
    met_count = sum(1 for v in criteria_met.values() if v)
    compliance_score = (met_count / criteria_count) * 60

    score['breakdown']['spec_compliance'] = compliance_score
    score['total'] += compliance_score

    unmet = [k for k, v in criteria_met.items() if not v]
    if unmet:
        score['feedback'].append(f"Not met: {', '.join(unmet)}. Adjust component valu

    # 3. Margin of Safety (20 pts)
    margins = validation_report['margin_of_safety']
    avg_margin = sum(margins.values()) / len(margins) if margins else 0

    if avg_margin < 5:
        margin_score = 5  # Too close to limits, risky
    elif avg_margin < 10:
        margin_score = 10
    elif avg_margin < 20:
        margin_score = 15
```

```
    else:
        margin_score = 20  # Safe margin

    score['breakdown']['margin_of_safety'] = margin_score
    score['total'] += margin_score

    if margin_score < 15:
        score['feedback'].append('Your design is close to the spec limits. Consider a

    # 4. Innovation (10 pts)
    efficiency = validation_report['component_efficiency']
    if efficiency > 0.95:
        innovation_score = 10
        score['feedback'].append('🌟 Excellent optimization! Minimal component waste.
    elif efficiency > 0.85:
        innovation_score = 7
    else:
        innovation_score = 4

    score['breakdown']['optimization'] = innovation_score
    score['total'] += innovation_score

    return score
```

Day 5: Database & Report Storage

- [ ] Store validation results in MongoDB
- [ ] Generate PDF report (auto-download)
- [ ] Publish results to student dashboard
- [ ] Send email notification with score

Success Metrics:

- 50+ circuits validated with detailed scoring
- Average final score distributed normally (mean ~72)
- Students receive actionable feedback
- PDF reports downloadable
- Leaderboard updated in real-time

WEEK 6-7: Frontend + Gamification

Week 6: Student Dashboard & Project UI

Deliverables:

Day 1-3: Student Onboarding

- [ ] Landing page (register / login)
- [ ] Domain selection (choose ECE)
- [ ] Psychometric assessment UI (interactive 30-min test)
- [ ] Profile creation with photo upload

Day 4-5: Project Discovery

- [ ] Project listing page
  - Filter by tier, difficulty, domain
  - Show psychology match score ("87% match — perfect for you!")
- [ ] Project detail page
  - Acceptance criteria, deliverables, timeline
  - Stipend display
  - Psychology match explanation
- [ ] Start project button → assignment

React Components:

```jsx
```

```jsx
// components/ProjectCard.jsx
import React from 'react';

export default function ProjectCard({ project, matchScore, matchReason }) {
  const getTierBadge = (tier) => {
    const badges = {
      'Trainee': '🎓',
      'Contributor': '⭐',
      'Associate': '💎',
      'Lead': '👑'
    };
    return badges[tier] || '🔰';
  };

  return (
    <div className="project-card">
      <div className="project-header">
        <h3>{project.title}</h3>
        <span className="tier-badge">{getTierBadge(project.requiredTier)}</span>
      </div>

      <div className="project-meta">
        <span className="difficulty">{project.difficulty}</span>
        <span className="duration">⏱️ {project.durationDays} days</span>
        <span className="stipend">₹{project.stipendRupees.toLocaleString()}</span>
      </div>

      <div className="psychology-match">
        <div className="match-score">
          {matchScore}% Match
          <div className="match-bar">
            <div className="match-fill" style={{width: `${matchScore}%`}}></div>
          </div>
        </div>
        <p className="match-reason">{matchReason}</p>
      </div>

      <div className="project-brief">
        {project.brief}
      </div>

      <button className="btn-start">
        Start Project →
```

```
      </button>
    </div>
  );
}
```

Week 7: Gamification & Blockchain Integration

Deliverables:

Day 1-2: XP System & Leaderboard

- [ ] XP calculation (points earned per project)
- [ ] Tier progression (Trainee → Contributor → Associate → Lead)
- [ ] Leaderboard (anonymized cohort ranking)
- [ ] Achievement badges

Day 3-4: Blockchain NFT Minting

- [ ] Smart contract deployment on Polygon Mumbai
- [ ] NFT metadata generation
- [ ] Automatic badge minting (quality_score > 75)
- [ ] Student wallet integration (Torus)

Day 5: Dashboard Polish

- [ ] My projects view (active, completed, in-review)
- [ ] Achievement/badge display
- [ ] NFT portfolio gallery
- [ ] XP progress tracker

Gamification Logic:

```javascript
```

```
// services/gamification.js

async function awardXPAndCheckTierPromo(submissionId) {
  const submission = await Submission.findById(submissionId);
  const project = await Project.findById(submission.projectId);
  const student = await Student.findById(submission.studentId);

  // Base XP = project value / 100
  // ₹8,000 project = 80 base XP
  let baseXP = Math.floor(project.stipendRupees / 100);

  // Quality multiplier
  const score = submission.qualityScore;
  let multiplier = 1.0;
  if (score >= 90) multiplier = 1.5;   // Excellent
  else if (score >= 80) multiplier = 1.3;  // Good
  else if (score >= 70) multiplier = 1.0;  // Pass
  else multiplier = 0.5;                   // Needs improvement

  // Speed bonus (if submitted before deadline)
  const daysLate = Math.max(0, (Date.now() - project.deadline) / (1000*60*60*24));
  const speedBonus = daysLate === 0 ? baseXP * 0.3 : 0;

  // Total XP
  const totalXP = Math.floor((baseXP * multiplier) + speedBonus);

  // Update student
  student.totalXP += totalXP;

  // Check tier promotion
  if (student.totalXP >= 500 && student.tier === 'Trainee') {
    student.tier = 'Contributor';
    await notifyPromotion(student, 'Contributor');
  }
  else if (student.totalXP >= 1500 && student.tier === 'Contributor') {
    student.tier = 'Associate';
    await notifyPromotion(student, 'Associate');
  }
  else if (student.totalXP >= 3500 && student.tier === 'Associate') {
    student.tier = 'Lead';
    await notifyPromotion(student, 'Lead');
  }
```

```
  // Mint NFT badge if score > 75
  let nftTxHash = null;
  if (score > 75) {
    nftTxHash = await mintNFTBadge({
      studentId: student._id,
      projectId: project._id,
      qualityScore: score,
      projectTitle: project.title
    });
  }

  await student.save();

  return {
    xpAwarded: totalXP,
    currentXP: student.totalXP,
    tier: student.tier,
    promoted: student.tier !== (await Student.findById(student._id)).tier,
    nftTxHash
  };
}
```

Success Metrics:

- 100+ students have completed projects
- XP distribution is visible on dashboard
- Leaderboard shows top 20 students
- 50+ NFT badges minted on Polygon
- Tier progression working (students reaching Contributor level)

WEEK 8: Beta Testing (50 ECE Users)

Deliverables:

Pre-Beta Checklist:

- [ ] Platform stability tested (load test 100 concurrent users)
- [ ] All user flows working end-to-end (register → project → submit → validate → NFT)
- [ ] Payment integration ready (though Tier 2/3 unlocks not monetized in MVP)
- [ ] Customer support system (Discord channel + email support)

Beta Launch:

- [ ] Recruit 50 ECE students (from 5 partner colleges)
- [ ] Recruit 5-10 ECE employers (startups, hardware companies)
- [ ] Daily standups to catch bugs
- [ ] Feedback surveys (NPS, feature requests)

Beta Week Schedule:

```
Day 1-2: Soft launch (invite 25 students)
Day 3-4: Ramp up (add 25 more students + 5 employers)
Day 5-7: Monitor & iterate (fix bugs, add features)

Metrics tracked:
- Daily active users (target: 40+)
- Project completion rate (target: >70%)
- Validation success rate (target: >95%)
- NPS score (target: >40)
- Churn rate (target: <5%)
```

Success Criteria:

- 40+ daily active users
- 50+ projects completed
- <2% critical bugs
- NPS > 40
- 3 paid employer commitments (for post-MVP)

WEEK 9: Polish & Optimization

Deliverables:

Day 1-3: Performance Optimization

- [ ] Page load time <2 seconds
- [ ] API response time <500ms
- [ ] Database query optimization (indexes, pagination)
- [ ] Frontend bundle size <300KB (gzipped)

Day 4-5: Bug Fixes & Polish

- [ ] Fix top 10 reported bugs
- [ ] Improve error messages (user-friendly)
- [ ] Add loading states & spinners

- [ ] Mobile responsiveness (tested on iOS/Android)
- [ ] Dark mode toggle

Optimization Checklist:

```javascript
// Performance monitoring (Sentry integration)
import * as Sentry from "@sentry/react";

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: "production",
  tracesSampleRate: 0.1,
  integrations: [
    new Sentry.Replay({ maskAllText: true, blockAllMedia: true })
  ]
});
```

Success Criteria:

- PageSpeed Insights score >85
- Lighthouse performance >90
- <100ms p95 API latency
- 0 critical bugs

WEEK 10: Public Launch + Investor Demo

Monday-Tuesday: Public Launch

Deliverables:

- [ ] Website live (talentforge.io)
- [ ] Press release (₹250 crore ECE talent market opportunity)
- [ ] Social media campaign (Twitter, LinkedIn, Instagram)
- [ ] Founder interviews (3 tech publications)

Press Release Headline:

```
"TalentForge Launches ECE Talent Marketplace
First Platform to Auto-Validate Circuit Designs,
Serves 800K Annual ECE Graduates in India"
```

Social Media Launch:

- Day 1: Founder announcement (personal story)
- Day 2-3: Student success story (video testimonial)
- Day 4-5: Employer ROI stats
- Day 6-7: Launch announcement + link

Target Reach:

- 5,000 organic signups (Week 10)
- 10,000 impressions on Twitter/LinkedIn
- 2-3 press mentions

Wednesday-Friday: Investor Presentation

Deliverables:

Investor Deck (12 slides):

Slide 1: Cover
"TalentForge ECE — India's First Performance-Verified Electronics Marketplace"

Slide 2: Problem
"800K ECE graduates/year. 65% unemployable due to lack of verified skills."

Slide 3: Solution
"Auto-validate circuit designs via SPICE simulation. Hire verified talent."

Slide 4: Product Demo
(Live demo or 2-min video)

Slide 5: POC Traction (Week 10)
- 300 students
- 50 employers
- ₹25L MRR
- 85% project completion rate
- 4.3 NPS

Slide 6: Unit Economics
- CAC (employer): ₹10K
- LTV (employer): ₹60L (24-month)
- LTV:CAC = 6:1 ✓
- Gross Margin = 75%

Slide 7: Market Opportunity
- TAM: ₹250 Cr (ECE only, India)
- SAM: ₹50 Cr (Year 3)
- By 2026: Add Mech, EEE, CS → ₹1,000 Cr TAM

Slide 8: Go-to-Market
- Phase 1: Scale to 10K students, 200 employers (3 months)
- Phase 2: Expand to EEE (parallel universe)
- Phase 3: Add Mechanical (Month 6)

Slide 9: Competitive Advantage
- Auto-validation (IP moat)
- Psychology matching (35% better fit)
- Blockchain credentials (portable)
- Multi-domain (we own all engineering)

Slide 10: Financial Projections
- Year 1: ₹35 Cr ARR

```
- Year 2: ₹246 Cr ARR (7x growth)
- Year 3: ₹480 Cr ARR

Slide 11: Team
- Founder: IIM MBA, ex-HackerRank
- CTO: 10 years embedded systems
- Advisor: NASSCOM

Slide 12: Ask
- ₹25 Cr Series A
- Valuation: ₹150 Cr (6x multiple on MRR)
```

Investor Presentation Materials:

- Executive summary (1-pager)
- Financial model (detailed P&L, unit economics)
- Customer testimonials (video + quotes)
- Media coverage clippings
- Technical architecture diagram
- Roadmap (next 12 months)

Success Metrics for Week 10:

- 300+ registered students
- 50+ employer signups
- ₹25L MRR (Month 10 run-rate)
- 5 case studies documented
- 3 press mentions
- 4.3+ NPS score
- Investor meetings scheduled

PART 3: TEAM STRUCTURE & HIRING

Core Team (10 weeks)

Role	Count	Monthly Cost	Responsibility
Full-Stack Engineers	3	₹6,00,000	Backend, Frontend, API, DB
Backend/SPICE Engineer	1	₹2,00,000	Circuit validator, simulation

Role	Count	Monthly Cost	Responsibility
DevOps/Infrastructure	1	₹1,50,000	AWS, Docker, CI/CD, monitoring
Product Manager	1	₹1,00,000	PRD, roadmap, user feedback
UI/UX Designer	1	₹80,000	Design system, wireframes, flows
QA/Tester	1	₹60,000	Test cases, bug hunting, validation
Community Manager	0.5	₹25,000	Discord, support, user engagement
Total	**8.5**	**₹11,15,000/month**	

10-Week Total Personnel Cost: ₹1,12,00,000 (₹11.2 Cr)

PART 4: INFRASTRUCTURE & BUDGET

Monthly Recurring Costs

Service	Cost	Notes
AWS	₹30,000	EC2, RDS, S3, Lambda
Polygon/Blockchain	₹5,000	Gas fees for NFT minting
Proteus/SPICE License	₹20,000	Cloud simulation access
Monitoring (DataDog)	₹10,000	Application monitoring
Email/SMS (SendGrid)	₹5,000	Notifications
Domain & SSL	₹2,000	talentforge.io
Slack/Collaboration	₹5,000	Team tools
****Subtotal/Month**	**₹77,000**	
10-week Total	**₹2,31,000**	

One-Time Setup Costs

Item	Cost
Smart contract audit	₹2,00,000
Legal & Compliance (ToS, Privacy)	₹1,00,000
Initial content creation (30 projects)	₹1,50,000
Marketing & PR launch	₹1,50,000
Total One-Time	**₹6,00,000**

TOTAL 10-WEEK POC BUDGET

```
Personnel:          ₹1,12,00,000  (64%)
Infrastructure:     ₹2,31,000     (1%)
Setup & Operations: ₹6,00,000     (3%)
Marketing/PR:       ₹1,50,000     (1%)
Contingency (10%):  ₹12,31,000    (7%)

TOTAL:              ₹1,34,12,000  (~₹1.34 Cr)

Simplified: ₹1.3-1.4 Crores for 10-week lean POC
```

PART 5: SUCCESS METRICS & KPIs

Week 10 Targets (POC Success Criteria)

Metric	Target	Status
Supply (Students)		
Registered	300+	
Active (weekly login)	200+	
Projects completed	200+	

Metric	Target	Status
Demand (Employers)		
Employers onboarded	50+	
Active projects posted	30+	
Conversions (hires)	10+	
Revenue		
MRR (Month 10 run-rate)	₹25L	
GMV (cumulative)	₹1.5Cr+	
Avg project value	₹15,000	
Quality		
Project completion rate	75%+	
Avg quality score	72/100	
Validation success rate	95%+	
User Satisfaction		
Student NPS	45+	
Employer NPS	40+	
Feature request volume	50+	
Technical		
Platform uptime	99.5%+	
Page load time	<2 sec	
API response time	<500ms	
Community		
College partnerships	8+	
Peer reviews submitted	100+	

Metric	Target	Status
Leaderboard engagement	70%+	

PART 6: RISK MITIGATION

Risk	Probability	Impact	Mitigation
SPICE simulation unstable	High	High	Use open-source ngspice + PySpice; test extensively with 100 circuits
Proteus API unavailable	Medium	High	Build fallback: accept SPICE netlists (.cir) directly from students
Low student adoption	Medium	High	Pre-commit 50 students via college partnerships
Employer hesitance	Medium	Medium	Offer first 5 projects free; provide detailed validation reports
Blockchain gas fees spike	Low	Low	Use Polygon (₹0.01 per tx); batch mints if needed
Data privacy issues	Low	Medium	GDPR/DPDP compliant; encrypt student data; regular audits

PART 7: GO-LIVE CHECKLIST

2 Weeks Before Launch:

- [ ] All 30 ECE projects loaded & tested
- [ ] 50 students + 20 employers in beta
- [ ] 0 critical bugs, <5 minor bugs
- [ ] Legal: ToS, Privacy Policy, T&Cs finalized
- [ ] Compliance: DPDP Act reviewed

1 Week Before Launch:

- [ ] Infrastructure load-tested (100 concurrent users)
- [ ] Database backed up
- [ ] Monitoring alerts configured

- [ ] Support process defined (Discord, email)
- [ ] Press release ready for media

Launch Day:

- [ ] Website live & DNS propagated
- [ ] Email notifications sent to beta users
- [ ] Social media posts scheduled
- [ ] Team on standby (Slack channel)
- [ ] Investor demo booked

FINAL: EXECUTION SUMMARY

10-Week Roadmap at a Glance

```
WEEK 1:    Setup, Architecture, Team
WEEK 2-3:  Backend Core (Auth, Users, Projects)
WEEK 4-5:  Circuit Validator (SPICE Integration)
WEEK 6-7:  Frontend Dashboard + Gamification
WEEK 8:    Beta Testing (50 users)
WEEK 9:    Polish & Optimization
WEEK 10:   Public Launch + Investor Demo

INVESTMENT:  ₹1.3-1.4 Crores
TEAM:        8-10 engineers + support
SUCCESS MET: 300 students, 50 employers, ₹25L MRR, 4.3 NPS
```

Post-POC Path to Series A

Month 3-4:

- Scale to 10,000 students, 200 employers
- ARR: ₹35+ Crores (annualized from MRR)
- Secure 5-10 corporate pilot contracts
- File patent for circuit validation system

Month 5-6:

- Launch EEE domain (parallel universe)
- Add Mechanical domain
- Secure NASSCOM partnership

Month 12:

- Series A ready: ₹50 Cr round at ₹150 Cr valuation
- ARR: ₹100+ Crores
- Team: 30+ people
- 3 engineering domains live

This POC proves the concept. Series A scales the execution.